

Funksionet në JavaScript — Dita 2

Parametrat, Return & Arrow Functions

Orën e kaluar mësuam çfarë janë funksionet, si i **deklarojmë** me `function`, si i **thërrasim** me `emri()`, dhe dallimin mes variablave **lokale** dhe **globale**. Por deri tani funksionet tona bënë gjithmonë të njëjtën gjë — printonin të njëjtin mesazh, çdo herë.

Sot i bëjmë më të fuqishme: do t'u japim **të dhëna** (parametra), do t'i lëmë të na **kthejnë rezultat** (`return`), dhe do të mësojmë dy mënyra të reja për t'i shkruar: **function expression** dhe **arrow functions**.

1. Funksionet me Parametra

Imagjinoje funksionin si një makinë: ia japim disa gjëra në hyrje, dhe ai punon me to. Ato "gjëra" quhen **parametra** dhe i shkruajmë brenda kllapave `()` të deklaramentit.

```
function sum(a, b) {  
  let result = a + b;  
  console.log(result);  
}
```

```
sum(3, 5);    // 8  
sum(10, 20); // 30
```

Çfarë ndodhi këtu?

- Kemi krijuar funksionin `sum` që pranon **2 parametra**: `a` dhe `b`.
- Brenda tij kemi një variabël `result` që ruan shumën e tyre.
- Kur e thërrasim, **duhet** t'i japim edhe numrat që duam t'i mbledhim: `sum(3, 5)`.

Këtë e filluam edhe në fund të orës së kaluar, me funksionin `mbledhja(a, b)`.

Parametra vs. Argumente

Dy fjalë që shpesh ngatërrohen, por kanë kuptim të ndryshëm:

```
function sum(a, b) { // a dhe b janë PARAMETRAT
  console.log(a + b);
}

sum(3, 5); // 3 dhe 5 janë ARGUMENTET
```

- **Parametrat** — emrat që shkruajmë kur **deklarojmë** funksionin (`a` , `b`).
- **Argumentet** — vlerat reale që japim kur e **thërrasim** funksionin (`3` , `5`).

💡 E bukura e parametrave: i njëjti funksion punon për shumë raste të ndryshme. Nuk na duhet një funksion për `3 + 5` dhe një tjetër për `10 + 20` — vetëm ndryshojmë argumentet.

⚠️ Nëse harrojmë të japim argumentet, parametrat mbeten `undefined`. P.sh. te `sum(3)` , `b` është `undefined` , kështu që `3 + undefined` na jep `NaN` .

💡 Parametrat sillen si **variabla lokale** — ekzistojnë vetëm brenda funksionit. Nëse provojmë ta printojmë `a` jashtë `sum` , do të marrim error (njësoj si `mesazhi` orën e kaluar).

2. `return` — Kthimi i një Rezultati

Deri tani funksionet tona thjesht **printonin** diçka. Por shpesh na duhet që funksioni të na **kthejë** një vlerë, që ta përdorim më tej në kod. Këtë e bëjmë me keyword-in `return` .

```
function sum(a, b) {
  let result = a + b;
  return result;
}

let total = sum(3, 5);
console.log(total); // 8
console.log(sum(10, 20)); // 30
```

Tani çdo thirrje e funksionit **kthen** vlerën `result` , dhe ne e ruajmë në një variabël (`total`) ose e printojmë direkt.

`console.log` **brenda funksionit vs.** `return`

Kjo është një nga gjërat më të rëndësishme për t'u kuptuar:

```
function shumaMeLog(a, b) {
  console.log(a + b); // vetëm PRINTON
}

function shumaMeReturn(a, b) {
  return a + b; // KTHEN vlerën
}

let x = shumaMeLog(3, 5); // printon 8
console.log(x); // undefined – nuk na ktheu asgjë!

let y = shumaMeReturn(3, 5); // nuk printon asgjë vetë
console.log(y); // 8
console.log(y * 2); // 16 – mund ta përdorim më tej
```

💡 `console.log` është vetëm për ta **parë** vlerën në console. `return` është për ta **marrë** vlerën dhe për ta përdorur përsëri (në një llogaritje, në një `if`, në një variabël tjetër).

`return` e ndalon funksionin

Kur ekzekutimi arrin te `return`, funksioni **ndalet aty** dhe vlera kthehet. Çdo gjë që vjen pas tij nuk ekzekutohet kurrë.

```
function test() {
  return "Përfundova";
  console.log("Ky rresht nuk ekzekutohet kurrë");
}

console.log(test()); // "Përfundova"
```

`return` bashkë me `if / else`

Këtë e përdorëm edhe në përgatitjen për Orën Sfiduese. Mund të kthejmë vlera të ndryshme varësisht një kushti:

```
function kontrolloRezultatin(nota) {  
  if (nota >= 5) {  
    return "Kaluar";  
  } else {  
    return "Nuk ka kaluar";  
  }  
}  
  
console.log(kontrolloRezultatin(7)); // "Kaluar"  
console.log(kontrolloRezultatin(4)); // "Nuk ka kaluar"
```

⚠ Nëse një funksion nuk ka `return`, ai kthen automatikisht `undefined`.

3. Function Declaration vs. Function Expression

Deri tani i kemi shkruar funksionet gjithmonë njësoj, me `function emri() { }`. Kjo quhet **Function Declaration**. Por ka edhe një mënyrë tjetër.

Function Declaration:

```
function pershendet(emri) {  
  return "Përshëndetje, " + emri;  
}
```

Function Expression — funksioni ruhet brenda një variable:

```
const pershendet = function (emri) {  
  return "Përshëndetje, " + emri;  
};
```

Të dyja thirren saktësisht njësoj:

```
console.log(pershendet("Arta")); // "Përshëndetje, Arta"
```

Në function expression, funksioni nuk ka emër të vetin (quhet **funksion anonim**) — emrin e merr nga variabla ku e ruajmë.

⚠ Kujdes: function expression mbyllet me `;` në fund, sepse është një deklaratim variable.

Dallimi i vërtetë: Hoisting

Një **function declaration** mund të thirret **para** se të deklarohet në kod, sepse JavaScript e "ngre" (hoist) lart para se të nisë ekzekutimin:

```
console.log(mbledh(2, 3)); // 5 ✓ punon, edhe pse funksioni është më poshtë

function mbledh(a, b) {
  return a + b;
}
```

Një **function expression** nuk punon kështu — ekziston vetëm **pasi** të ekzekutohet rreshti ku është deklaruar:

```
console.log(shumezo(2, 3)); // ✗ Error

const shumezo = function (a, b) {
  return a * b;
};
```

💡 Rregull i thjeshtë: me function expression, fillimisht **deklarojmë**, pastaj **thërrasim**.

4. Arrow Functions

Arrow functions janë një mënyrë më e shkurtër për të shkruar funksione. Në vend të keyword-it `function`, përdorim shigjetën `=>`.

Le ta shohim hap pas hapi, duke nisur nga function expression që e njohim:

```
// 1. Function expression
const sum = function (a, b) {
  return a + b;
};
```

```
// 2. Arrow function – heqim "function", shtojmë "=>"
const sum = (a, b) => {
  return a + b;
};
```

```
// 3. Arrow expression – kur trupi është vetëm një shprehje,  
// heqim edhe { } dhe return (kthehet automatikisht)  
const sum = (a, b) => a + b;
```

E përdorim njësoj si çdo funksion tjetër:

```
console.log(sum(3, 5)); // 8
```

Variacionet e sintaksës

```
// Një parametër – kllapat janë opsionale  
const dyfisho = numri => numri * 2;  
  
// Pa parametra – kllapat bosh janë të detyrueshme  
const pershendet = () => "Përshëndetje!";  
  
// Disa rreshta – na duhen { } dhe return i qartë  
const kontrolloRezultatit = (nota) => {  
  if (nota >= 5) {  
    return "Kaluar";  
  } else {  
    return "Nuk ka kaluar";  
  }  
};
```

⚠ Gabimi më i zakonshëm: nëse përdorim `{ }`, **duhet** të shkruajmë `return` vetë. Pa të, funksioni kthen `undefined`. P.sh. `const sum = (a, b) => { a + b };` dhe pastaj `sum(3, 5)` na kthen `undefined`, jo `8`.

💡 Arrow functions, si function expressions, **nuk** bëjnë hoisting – duhen deklaruar para se të thirren.

Ushtrimet që bëjnë në klasë

Detyra 1 – Prodhimi i tre numrave

Krijoni një funksion që merr 3 parametra (numra) dhe i shumëzon ata ndërmjet vete për të arritur prodhimin final.

```
function prodhimi(a, b, c) {  
  let rezultati = a * b * c;  
  console.log(rezultati);  
}  
  
prodhimi(2, 3, 4); // 24
```

Detyra 2 — Prezantimi

Krijoni një funksion që merr 3 parametra (emrin, mbiemrin, moshën) dhe printon fjalinë: "Unë jam Filan Fisteku dhe jam X vjeçarë."

```
function prezantohu(emri, mbiemri, mosha) {  
  console.log("Unë jam " + emri + " " + mbiemri + " dhe jam " + mosha + " vjeçarë.");  
}  
  
prezantohu("Filan", "Fisteku", 25);  
// Unë jam Filan Fisteku dhe jam 25 vjeçarë.
```

💡 Për ushtrim në shtëpi: provoni t'i rishkruani të dyja detyrat me `return` në vend të `console.log`, dhe pastaj si arrow functions.

Përmbledhje e shpejtë

1. function emri(p1, p2) { }	→ p1, p2 janë PARAMETRAT (në deklarim)
2. emri(arg1, arg2)	→ arg1, arg2 janë ARGUMENTET (në thirrje)
3. return vlera;	→ e kthen vlerën dhe e ndalon funksionin
4. console.log ≠ return	→ printon vs. kthen vlerë që mund ta përdorim
5. const f = function () { };	→ function expression (nuk bën hoisting)
6. const f = (a, b) => a + b;	→ arrow function (return automatik pa { })
7. Pa return	→ funksioni kthen undefined

Fjalë të shkurtra

- **Parametër** — emri i variablës brenda kllapave kur deklarojmë funksionin
- **Argument** — vlera reale që japim kur thërrasim funksionin

- **return** — keyword që kthen një vlerë nga funksioni dhe e ndalon ekzekutimin e tij
- **Function Declaration** — funksion i shkruar me `function emri() { }`
- **Function Expression** — funksion i ruajtur brenda një variable: `const f = function () { }`
- **Funksion anonim** — funksion pa emër të vetin
- **Hoisting** — sjellja e JavaScript-it që "ngre" function declarations në fillim, kështu që mund të thirren para deklarimit
- **Arrow Function** — sintaksë e shkurtër për funksione, me shigjetën `=>`

Sfida jote (pikë ekstra)

Shkruaj një funksion `mesatarja(n1, n2, n3)` që **kthen** (`return`) mesataren e tri notave.

Pastaj shkruaj një **arrow function** `vlereso(mesatarja)` që kthen (për pikë të plotë, shkruaj edhe `mesatarja` si arrow function me një rresht):

- Mbi **4.5** → "Sukses të shkëlqyeshëm"
- Mbi **3.5** (deri 4.5) → "Sukses shumë të mirë"
- Për çdo rast tjetër → "Duhet më shumë punë"

Thirri të dyja bashkë:

```
console.log(vlereso(mesatarja(5, 4, 5)));
```